

tmux documentation

Working Plan — v1.0.5 cycle (2026-05-21)

Revision: 1 **Last modified:** 2026-05-21T06:00:00Z **Scope:** every Linux-only gate gains a macOS-equivalent implementation per the user mandate 2026-05-21. Tests dispatch on `uname -s` and execute the platform-correct invariant. §11.4.6: where XNU genuinely cannot enforce the Linux primitive, the test SKIPS with verbatim kernel-citation evidence AND a sibling test exercises what XNU CAN enforce.

§1 Triggering directive (verbatim, 2026-05-21)

"Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

§2 Platform-equivalence matrix

Linux primitive	macOS equivalent	Action this cycle
<code>/proc/<pid>/oom_score_adj</code> (test 08)	macOS has NO OOM killer; rlimit wrapper bounds the session instead (tested by test 15)	Test 08 keeps SKIP-with-kernel-citation on Darwin; verify-only output cites the §5.6 kernel gap doc
<code>systemd-run --user --scope</code> (test 09)	<code>scripts/tmx-rlimit-wrapper.sh</code> (POSIX rlimit)	NEW Darwin branch in test 09: prove rlimit wrapper invoked + <code>RLIMIT_CPU</code> + <code>RLIMIT_NPROC</code> applied + server isolated
cgroup MemoryMax enforcement (test 12)	XNU does NOT enforce <code>RLIMIT_AS</code> for unprivileged → kernel gap	Test 12 keeps SKIP-with-kernel-citation; NEW test 24 exercises macOS <code>RLIMIT_CPU+SIGXCPU</code> which IS enforced
cgroup TasksMax (test 13)	<code>RLIMIT_NPROC</code> (per-user, per-process)	NEW Darwin branch in test 13: spawn processes until <code>RLIMIT_NPROC</code> hit; assert next fork fails
3-scope OOM independence (test 14)	3 sessions, direct kill of server A, verify B+C servers + sessions survive	NEW Darwin branch in test 14: same operator-path; direct <code>kill -9</code> instead of OOM
M4 strip <code>systemd-run</code> from wrapper	Mutation correctly SKIP-with-topology-reason on Darwin (target unreachable on Darwin code path)	Keep as-is — §11.4.3 honest dispatch
M5 strip MemoryMax from wrapper	Same as M4	Keep as-is
M7-M10 target <code>scripts/tmx-vm</code>	DEAD CODE — VM path retired in Fixed.md A4-A8 native dual-OS	RETIRE M7-M10 + replace with Darwin-equivalent mutations against <code>tmx-rlimit-wrapper.sh</code>

Linux primitive	macOS equivalent	Action this cycle
New Darwin mutations	—	M20 strip <code>ulimit -t</code> from <code>rlimit wrapper</code> → test 15 T5 FAILs (CPU cap not enforced); M21 strip <code>ulimit -u</code> from <code>rlimit wrapper</code> → test 15 T3 FAILs (NPROC not visible)

§3 Phased implementation

Phase 1 — Test 09 Darwin branch

- Add case "`$(uname -s)`" in Darwin) ... esac block that:
 - Spawns 2 sessions via `tmx new -s isol1 -d + tmx new -s isol2 -d`
 - Reads back `ulimit -t` (CPU) + `ulimit -u` (NPROC) inside each session via `send-keys/capture-pane`
 - Asserts both values match the wrapper's defaults (86400s, 2666 procs derived from host)
 - SIGKILL session-1's tmux server directly; verify session-2's server survives with original PID
 - Captured evidence: `ulimit` readback per session + `ps` before/after kill

Phase 2 — Test 13 Darwin branch

- Spawn a session, then a child shell with low `RLIMIT_NPROC` (e.g., 64)
- Fork until limit hit; assert `fork()` returns `EAGAIN`
- Captured evidence: actual fork count = `NPROC` - already-running pids (positive evidence per §11.4.5)

Phase 3 — Test 14 Darwin branch

- 3 sessions A/B/C via `tmx new`
- Capture each server's PID (tmx-LABEL socket)
- Direct kill server A (SIGKILL)
- Assert B + C servers still running with ORIGINAL PIDs
- Assert `tmx ls` no longer shows A

Phase 4 — NEW Test 24 (Darwin CPU-cap enforcement, macOS-only)

- Spawn a session with `TMX_CPU_HARD_SEC=5`
- Inside the session, run a CPU-bound loop
- Assert process is SIGXCPU'd ≤ 7 seconds (5s soft + epsilon)
- Captured evidence: process exit signal + wall-clock duration
- §11.4.3 dispatch: SKIPs on Linux (test 12 covers Linux memory pressure; Linux CPU cap is via cgroup)

Phase 5 — Retire M7-M10; add M20 + M21 Darwin rlimit mutations

- M7-M10 explicitly retired with comment block citing native-dual-OS dead-code rationale
- M20: strip `ulimit -t` line from `scripts/tmx-rlimit-wrapper.sh` → test 15 T5 FAILs (`TMX_CPU_HARD_SEC` override no longer visible)
- M21: strip `ulimit -u` line from `scripts/tmx-rlimit-wrapper.sh` → test 15 T3 FAILs (`TMXLIMITS:cpu=...:nproc=` no longer carries `nproc`)

Phase 6 — Challenges + docs + CONTINUATION

- TMUX-CH-24 for the new CPU-cap test
- docs/guide/README.md §5.6 — refresh isolation-strength matrix to reflect Darwin-branch test coverage
- docs/plans/v1.0.5.md — this plan
- Fixed.md A22-A26 entries
- CHANGELOG.md v1.0.5 block
- CONTINUATION.md §3.11 + §1 snapshot bump
- VERSION 1.0.5 / 6

Phase 7 — Verification + commit + push

- Fresh suite + meta-test + e2e
- §11.4.71 fetch + integrate
- commit_all.sh

§4 Anti-bluff invariants (verbatim user mandate, 2026-05-21)

"all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used!"

Every Darwin branch added in this cycle MUST:

1. Read CONTENT (ulimit values, ps output, fork return codes), not file existence.
2. Use the operator path (tmx new -s NAME -d) per §102.
3. SKIP-with-reason where macOS genuinely can't (kernel gap), citing the exact rlimit + reproducer per §11.4.6 no-guessing.
4. Have a paired §103 layer-4 mutation in the meta-test.

§5 Out-of-scope (logged for future cycles)

- Linux remote-host runner — to actually exercise the Linux branches of tests 09/13/14 from a Darwin dev host, we'd need a Linux test target. This cycle ensures both branches are CODED correctly; Linux-side validation lands when a Linux CI host is provisioned.
- macOS launchd job-based hard memory cap (would need root) — out of scope per §11.4.10 + §11.4.78 no-sudo philosophy.